

الأسئلة النظرية (Part 01)

1. توفر خوارزميات الفرز الـ Generic أماناً للأنواع (Type Safety) أثناء وقت الترجمة، مما يمنع أخطاء الـ Runtime، كما أنها تزيد من إعادة استخدام الكود وتمنع مشكلة الـ Boxing و Unboxing مع الـ Value Types.
1. تعمل تعبيرات اللامدا (Lambda Expressions) على تحسين قابلية القراءة من خلال السماح بكتابة منطق الفرز المخصص في سطر واحد مباشرة في مكان استدعاء الدالة، مما يجعل الكود أكثر مرونة وأقل تعقيداً.
1. من المهم استخدام دوال مقارنة ديناميكية لأن الأنواع المختلفة تتطلب معايير فرز مختلفة (مثل فرز النصوص بالطول أو الموظفين بالراتب). هذا يبيّن دالة الفرز عامة وقابلة للتطبيق على أي نوع.
1. تنفيذ `IComparable<T>` في الكلاسات المشتقة يوفر طريقة قياسية وافترضية لمقارنة الكائنات ببعضها، مما يسمح لدوال الفرز بالعمل عليها تلقائياً بناءً على المنطق المخصص بداخلها (مثل الراتب).
1. استخدام الـ Delegates المدمجة مثل `Func` و `Action` يوفر وقت كتابة تعريفات جديدة للـ Delegates، ويجعل الكود قياسياً ومفهوماً لأي مطور `C#`، مما يسهل البرمجة العامة.
1. الدوال المجهولة (Anonymous Methods) وتعبيرات اللامدا يخدمان نفس الغرض، لكن اللامدا أكثر إيجازاً وقابلية للقراءة. من حيث الكفاءة هما متطابقان تقريباً حيث يتم تحويلهما لنفس الكود وقت الترجمة.
1. الطرق العامة (Generic Methods) مفيدة جداً في دوال مثل `Swap` لأنها تتيح لك كتابة كود التبدل مرة واحدة فقط، وسيعمل مع أي نوع بيانات (أرقام، نصوص، كائنات) بدلاً من تكرار الكود.
1. فوائد الفرز متعدد المعايير هي القدرة على تنفيذ قواعد عمل معقدة، أما التحديات فتكمن في زيادة تعقيد كود المقارنة والحاجة لضمان خلوه من الأخطاء المنطقية أثناء التعامل مع القيم المتساوية.
1. الكلمة المفتاحية `default(T)` ضرورية لأننا لا نعرف نوع `T` مسبقاً. فهي ترجع القيمة الافتراضية المناسبة: `null` للـ `Reference Types`، و `0` أو `false` للـ `Value Types`.
1. تضمن القيود (Constraints) مثل `where T : IComparable` أن النوع المُمرر يحتوي على الخصائص المطلوبة (مثل دالة المقارنة)، مما يمنع الأخطاء أثناء التنفيذ ويزيد من موثوقية الكود.
1. استخدام الـ Delegates في تحويل النصوص يفصل عملية التكرار (Looping) عن عملية التحويل نفسها، مما يدعم مفهوم البرمجة الوظيفية ويجعل الكود أكثر تركيبية وإعادة استخدام.
1. يعزز استخدام الـ Delegates من مرونة العمليات الحسابية، حيث يمكنك كتابة دالة واحدة تستقبل الأرقام ونوع العملية (جمع، طرح..). كـ `Delegate`، مما يسهل إضافة عمليات جديدة دون تعديل الدالة الأصلية.
1. تفصل الـ Generic Delegates خوارزمية التحويل عن أنواع البيانات، مما يتيح لك تحويل أي قائمة من نوع `T` إلى نوع `R` بمرونة تامة دون كتابة كود مخصص لكل حالة.

1. يبسط Func إنشاء الـ Delegates لأنه يوفر قوالب جاهزة تستقبل مدخلات وتُرجع قيمة، مما يغنيك عن تعريف Delegates مخصصة في كل مرة.
1. يُفضل استخدام Action للعمليات التي لا تُرجع قيمة (void) لأنها تعبر بوضوح عن الغرض من الكود وهو تنفيذ إجراء معين (مثل الطباعة) دون انتظار نتيجة.
1. تلعب الـ Predicates دوراً أساسياً في الفلترة وتحديد الشروط، فهي تجعل شروط الكود واضحة ومقروءة كأنها جملة منطقية بسيطة تعيد True أو False.
1. تعمل الدوال المجهولة على تحسين نمطية الكود (Modularity) بتغليف المنطق البسيط في مكان استخدامه فقط، دون الحاجة لإنشاء دوال منفصلة تزيد من زحمة الكود.
1. يُفضل استخدام الدوال المجهولة عندما تكون العملية الحسابية بسيطة وتُستخدم لمرة واحدة فقط في هذا السياق المحدد، بينما تُفضل الدوال المسماة إذا كان المنطق سيتكرر.
1. تعتبر تعبيرات اللامدا ميزة أساسية لأنها حجر الأساس للـ LINQ والبرمجة الوظيفية في C#، وتسمح بكتابة كود قوي ومختصر جداً.
1. تعزز اللامدا من التعبير عن العمليات الحسابية من خلال كتابتها بشكل يشبه المعادلات الرياضية البسيطة (مثل $(a, b) \leq a / b$) مما يجعل الكود بديهياً وسهل الفهم.

ثانياً ::

-2

- **البرمجة: (والتزامن المتوازية البرمجة) Parallel Programming and Concurrency**
المعالج أنوية تعدد من مستفيدة الوقت نفس في تعمل أصغر مهام إلى الكبيرة المهمة تقسيم تعني المتوازية التنفيذ وقت لتقليل الثقيلة الحسابية للعمليات ممتاز الأسلوب هذا (Multi-core processors). أو بياناتها تتداخل أن دون متداخلة متعددة مهام إدارة على النظام قدرة يعني (Concurrency) التزامن مشاكل تحدث.
- **Unit Testing and TDD (بالاختبار الموجه والتطوير الوحدات اختبار)**
هو الـ Unit Testing. أما بكفاءة تعمل أنها من للتأكد الأساسي المشروع كود من (وحدات) صغيرة أجزاء يختبر كود كتابة أن يضمن مما نفسه؛ التطبيق كود كتابة قبل "أو لا الاختبار كتابة" على تعتمد تطوير منهجية فهو TDD بالضبط المتطلبات ويلبي الأخطاء من خال النهائي الكود.
- **Asynchronous Programming (المتزامنة غير البرمجة)**
يسمح await و async استخدام (المتزامنة غير البرمجة). بيانات بقاعدة الاتصال أو ملف تحميل مثل طويلة مهام انتظار أثناء (Blocking) التجمد بعدم للتطبيق لتكملة يعود، الطويلة المهمة تنتهي وعندما، أخرى مهام تنفيذ في (Thread) الخيط يستمر، ذلك من بدلا واستجابته التطبيق أداء يحسن مما، بها الخاص الكود.

رابعاً: Part 03

1- تقرير الدراسة الذاتية (Self Study Report)

خلال هذا التكليف، ركزت دراستي على فهم التطبيقات المتقدمة في C# وتحديد الـ Generics والـ Delegates. قمت بتطبيق طرق الفرز (Sorting) باستخدام الـ Generics لضمان الـ Type Safety وكيفية إضافة Constraints لضمان دقة أنواع البيانات المدخلة. بالإضافة إلى ذلك، تدربت بشكل مكثف على استخدام Func، Action، و Predicate كبدايل حديثة ومختصرة للـ Delegates التقليدية، وكيفية تمرير الدوال

كمعاملات (Parameters) باستخدام تعبيرات اللامدا والدوال المجهولة للوصول إلى كود مرّن يعتمد على الـ Functional Paradigm.

2- ما هي البرمجة غير المتزامنة (Asynchronous Programming)؟

هي تقنية برمجية تسمح للبرنامج ببدء عملية طويلة (مثل الاستعلام من داتا بيز أو جلب بيانات من API) دون إيقاف باقي البرنامج لانتظار هذه العملية حتى تنتهي. بفضل الكلمات المفتاحية `await` و `async`، يتم تحرير الـ Thread الأساسي للقيام بمهام أخرى، مما يحافظ على استجابة النظام وسرعته، وبمجرد انتهاء المهمة الطويلة يعود التطبيق لاستلام النتيجة وإكمال العمل عليها.